



Instituto Politécnico Nacional

Escuela Superior de Cómputo

Practica 1

Investigación de Bases de Datos y Caso de Estudio

Materia: Circuitos Eléctricos

Profesor: Gabriel Hurtado Avilés

Alumnos:

- Damas Garnica Adrian
-
-

Grupo: 3CV4

Resumen

En este documento se presenta la documentación inicial del desarrollo de una base de datos para el desarrollo de dicha base de datos focalizada en un sistema de comandas.

Índice

1	Introducción	1
1.1	Planteamiento del problema	2
1.2	Justificación	2
1.3	Objetivo general	3
1.4	Objetivos específicos	3
1.5	Estado del arte	4
1.5.1	Análisis del estado del arte	5
1.6	Productos esperados	6
2	Caso de Estudio	7
2.1	Entrevista con el Cliente	7
2.1.1	Eventos y mesas	7
2.1.2	Menú y productos	7
2.1.3	Flujo de comandas	8
2.1.4	Cobros y pagos	9
2.1.5	Roles y permisos	9
2.1.6	Reportes	9
2.1.7	Operación y escalabilidad	10
2.2	Requerimientos de la base de datos	11
2.2.1	Entidades y atributos	11
2.2.2	Relaciones	11
2.3	Requerimientos del sistema en general	12
2.4	Diagrama Entidad-Relación	13
2.5	Justificación del modelo	14
2.5.1	Justificación de las entidades	14
2.5.2	Justificación de las relaciones	15
2.5.3	Cómo resuelve el modelo la problemática planteada	16
3	Conclusiones	18
4	Anexos	20

1 Introducción

En la siguiente sección se abarca la investigación previa para el desarrollo de la base de datos como proyecto final de la materia Bases de Datos, se plantean las bases para el proyecto y el producto esperado de esta investigación.

1.1 Planteamiento del problema

Los salones de eventos que ofrecen servicio de banquetes suelen gestionar la toma y el seguimiento de comandas –los pedidos de alimentos y bebidas realizados por cada mesa o por cada invitado durante un evento– de forma manual, apoyándose en comandas de papel, hojas de cálculo o aplicaciones genéricas que no fueron diseñadas para este giro de negocio. Esta forma de operar genera problemas concretos: la información de los pedidos se duplica o se pierde entre el área de meseros, cocina y caja; no existe un registro histórico confiable de qué se sirvió en cada evento, a qué cliente y a qué costo; el control de existencias de insumos se lleva de manera aproximada, lo que provoca faltantes o compras innecesarias; y la elaboración de la cuenta final de cada evento es lenta y propensa a errores porque depende de conciliar manualmente varias comandas en papel.

Este proyecto retoma y da continuidad a un desarrollo realizado en un semestre anterior, en el que se propuso un primer sistema de comandas de uso general. Sin embargo, aquella versión inicial se apoyaba en un sistema enteramente en C y este únicamente trabajaba con archivos en formato ASCII para almacenar la información, esto causa que dicho sistema tenga relaciones mal definidas, escasa normalización y ausencia de reglas de integridad, lo que dificultaba su mantenimiento y su crecimiento. Por ello, el problema que atiende la presente práctica no es únicamente digitalizar la toma de comandas, sino rediseñar, con una metodología formal de modelado entidad-relación, la base de datos que sustenta dicho sistema, de modo que sea capaz de representar correctamente eventos, clientes, mesas, menús, platillos, insumos, meseros y pagos, garantizando consistencia, integridad y escalabilidad de la información, de esta forma concretar con un sistema profesional y funcional para nuestro proyecto final de la materia de Bases de Datos.

1.2 Justificación

El diseño de una base de datos robusta para un sistema de comandas resulta pertinente porque este tipo de sistema constituye el núcleo operativo de un salón de eventos: de él dependen la asignación de mesas, el control de los pedidos por evento, la disponibilidad de insumos y, en última instancia, la facturación al cliente. Un modelo de datos mal estructurado no solo genera inconsistencias en la información, sino que limita la capacidad del sistema para escalar a medida que el negocio crece o para adaptarse a nuevos requerimientos, como el manejo de menús personalizados, distintos tipos de evento o reportes administrativos.

Desde el punto de vista académico, esta práctica es relevante porque permite aplicar de forma integral los conceptos revisados a la fecha en la materia de Bases de Datos –definición de una base de datos, implementación de una base de datos, modelos entidad relación, uso de docker y uso de PostgreSQL– a un caso de estudio real y ya conocido por el equipo, lo que facilita identificar con claridad los errores de diseño cometidos previamente y corregirlos mediante un modelo entidad-relación formal, con entidades, atributos, relaciones y cardinalidades correctamente definidas.

Desde el punto de vista práctico, contar con una base de datos bien diseñada beneficia directamente la operación de un salón de eventos: reduce el tiempo de atención al reducir errores en la toma de comandas, permite conocer con precisión el costo y las existencias de insumos por evento, y facilita la generación de reportes confiables para la toma de decisiones administrativas, sentando además una base sólida sobre la cual continuar el desarrollo del sistema en etapas posteriores del proyecto.

1.3 Objetivo general

Diseñar el modelo entidad-relación de una base de datos para un sistema de comandas de un salón de eventos, que permita gestionar de manera íntegra y consistente la información de clientes, eventos, mesas, menús, platillos, insumos, meseros y pagos, con el fin de sustentar una versión mejorada del sistema desarrollado previamente.

1.4 Objetivos específicos

1. Investigar y comparar la definición formal de dato, información y base de datos, así como las características fundamentales de un sistema gestor de bases de datos.
2. Identificar y documentar, mediante una entrevista con el cliente, los requerimientos de información y las funciones que debe cumplir el sistema de comandas del salón de eventos.
3. Analizar las deficiencias de diseño presentes en la base de datos del proyecto realizado en el semestre anterior, con el fin de evitarlas en la nueva propuesta.
4. Construir el diagrama entidad-relación del sistema de comandas, incluyendo entidades, atributos, relaciones y cardinalidades.
5. Justificar las decisiones de diseño del modelo propuesto, explicando cómo cada entidad y relación contribuye a resolver la problemática planteada.

1.5 Estado del arte

Referencia	Problema y metodología	Resultados principales	Relación con la Unidad Temática I	Relación con el proyecto
Aguilar Vera et al. (2023)	Los autores realizan una revisión sistemática de la literatura sobre el modelado y la gestión de bases de datos NoSQL. Analizan los procesos de modelado, los diferentes tipos de datos y las herramientas disponibles para su administración.	Se identifican cuatro modelos principales de bases de datos NoSQL: clave-valor, documentos, columnas y grafos. El modelo orientado a documentos destaca por su flexibilidad y uso frecuente.	El artículo se relaciona con la unidad temática 1.4, «Modelos de datos», al ampliar la clasificación vista en clase.	Para el sistema de comandas, permite contrastar el uso de NoSQL con el modelo relacional requerido para manejar cuentas, pagos y comandas.
Taipalus (2024)	Presenta una revisión de 117 estudios que comparan el desempeño de 44 sistemas gestores de bases de datos. También analiza el rigor metodológico y la posibilidad de reproducir los resultados de dichas comparaciones.	El estudio señala que existen pocas comparaciones de alto rigor y reproducibilidad entre SGBD. En los estudios analizados, los sistemas NoSQL presentan, en general, un mejor desempeño que los sistemas NewSQL en operaciones de lectura y escritura.	El artículo se relaciona con la unidad temática 1.2, «Sistema gestor de base de datos», especialmente con su clasificación.	En el proyecto, sirve como referencia para analizar la elección de PostgreSQL considerando no solo el desempeño, sino también las necesidades de transacciones e integridad de los datos.
Machuca & Braul (2022)	Los autores realizan una revisión de 14 artículos publicados entre 2016 y 2021 para identificar las metodologías más utilizadas en la seguridad de bases de datos. Las fuentes consultadas incluyen IEEE, arXiv, Hindawi y Dialnet, tras la revisión se seleccionaron 14 artículos para ser analizados.	El control de acceso es la estrategia de seguridad más utilizada en los trabajos revisados, seguido por técnicas de cifrado y mecanismos de protección para bases de datos en la nube.	Se relaciona con la unidad temática 1.1.1 «Características de una base de datos», en concreto Seguridad e Integridad.	Se relaciona con las características de seguridad e integridad de las bases de datos. Para el sistema de comandas, respalda la implementación de diferentes roles y permisos para usuarios como meseros, personal de cocina, bar y cajeros o administradores.

Tabla 1
Comparación de trabajos relacionados con el proyecto

1.5.1 Análisis del estado del arte

Al revisar los tres trabajos, lo primero que se nota es que ninguno propone un sistema nuevo. Los tres son revisiones sistemáticas de literatura, así que su aporte consiste en reunir y ordenar lo que otros autores ya han publicado para mostrar cómo se encuentra cada área. Aunque tratan temas distintos, parten de una preocupación parecida: las aplicaciones actuales manejan datos cada vez más variados, exigen un mejor desempeño y necesitan proteger la información frente a accesos no autorizados. Estos problemas son los mismos que dan sentido a los contenidos de la Unidad Temática I, lo cual indica que conceptos como modelo de datos o sistema gestor de bases de datos siguen siendo el punto de partida para estudiar las tecnologías actuales.

Cada artículo, sin embargo, mira el problema desde un ángulo diferente. Aguilar Vera et al. (2023) se ocupan del modelado y la clasificación de las bases de datos NoSQL, con lo que amplían lo visto en el apartado 1.4, «Modelos de datos». Taipalus (2024) compara el desempeño de distintos sistemas gestores, tema que corresponde al apartado 1.2, «Sistema gestor de base de datos». Por último, Machuca y Braul (2022) revisan las estrategias de seguridad más empleadas, entre ellas el control de acceso, el cifrado y la protección de bases de datos en la nube, y su trabajo se vincula con las características de seguridad e integridad del apartado 1.1.1. Leídos en conjunto, los tres artículos dejan ver que diseñar una base de datos implica bastante más que elegir un modelo o un gestor; también hay que considerar cómo responderá el sistema ante la carga de trabajo y quién podrá consultar o modificar cada dato.

Otro punto en común es que ninguna de las tres áreas cuenta con criterios del todo estandarizados. En NoSQL conviven varios modelos y diversas formas de modelar. En las comparaciones de desempeño, Taipalus (2024) encuentra pocos estudios rigurosos y reproducibles, además de metodologías que varían mucho entre sí. En seguridad tampoco hay un enfoque único que funcione para todos los escenarios. De aquí se desprende que no existe una tecnología de base de datos superior en términos absolutos y que la elección debe partir de las necesidades de cada sistema.

Esta idea orienta las decisiones de diseño de nuestro proyecto. La revisión de Taipalus (2024) reporta diferencias de desempeño importantes entre familias de gestores, con los sistemas NoSQL por encima de los NewSQL en operaciones de lectura y escritura; sin embargo, en un sistema de comandas la velocidad no es el criterio que más pesa. Una comanda duplicada, un pago mal registrado o una cuenta que no cuadra al cierre de un evento afectan directamente la operación del salón y la confianza del cliente. Por esa razón se da prioridad a la integridad y la consistencia de los datos, y PostgreSQL, al ser un gestor relacional con soporte para transacciones y restricciones de integridad, resulta la opción más adecuada. De manera similar, lo reportado por Machuca y Braul (2022) respalda la definición de roles y permisos diferenciados, de modo que meseros, personal de cocina y bar, cajeros y administradores accedan solo a la información que les corresponde. En resumen, estos trabajos complementan lo estudiado en la Unidad Temática I y, al mismo tiempo, aportan argumentos concretos para sustentar el diseño del sistema.

1.6 Productos esperados

Como resultado de esta práctica se espera obtener un conjunto de entregables documentales y técnicos que evidencien tanto el proceso de investigación como el diseño de la base de datos del sistema de comandas.

Los productos esperados son:

- Descripción del caso de estudio, incluyendo la entrevista con el cliente y la lista de requerimientos del sistema de comandas.
- Diagrama entidad-relación del sistema de comandas para el salón de eventos, con entidades, atributos, relaciones y cardinalidades.
- Las bases de la implementación de la base de datos para el proyecto final.

2 Caso de Estudio

En esta sección se abarca la problemática a trabajar a lo largo del semestre para culminarlo en el proyecto final de la materia de Bases de Datos. La problemática que abarcaremos será un sistema de comandas, siendo nuestro primer cliente un salón de eventos enfocándose en el control del flujo de trabajo y el inventario de la mercancía durante los evento, además de mantener controlado pagos y estados de pedidos; por lo tanto, la primera instancia del diseño de nuestro sistema estará enfocada en atender las necesidades de este primer cliente.

Además, se ha de aclarar que este proyecto ya tiene un sistema inicial de apoyo, siendo una *interfaz de usuario en terminal* (TUI) hecha con `ncurses`, la cual se desarrolló en la materia de Sistemas Operativos.

2.1 Entrevista con el Cliente

2.1.1 Eventos y mesas

- ¿El salón atiende uno o varios eventos simultáneamente?

R: Se suele atender un único evento a la vez.

- ¿Cuántas mesas suelen usar por evento?

R: El número varía, pero suelen ser alrededor de 20 mesas.

- ¿Esa cantidad varía entre eventos?

R: Sí.

- ¿Las mesas están numeradas?

R: Sí, cada mesa tiene su número.

- ¿Todos los meseros atienden todas las mesas o se le asigna a cada mesero un número de mesas?

R: Dependiendo del evento; a veces solo hay un mesero, pero de ser múltiples se asigna a cada mesero un grupo de mesas.

2.1.2 Menú y productos

- ¿El menú es un todo incluido por evento o se atiende a la carta?

R: La carta es aparte; en los eventos se puede pedir comida preparada con antela-

ción, pero siempre está la opción de la carta.

- ¿Qué categorías de productos manejan?

R: Alimentos, bebidas con y sin alcohol, y dulcería.

- ¿Los productos tienen precio fijo?

R: Durante el evento sí, pero tienen tendencia a cambiar por temporada.

- ¿Se manejan variantes de platillos?

R: Sí, algunas bebidas o platillos tienen tamaños distintos o preparaciones distintas.

- ¿Se necesitan registrar ingredientes o alérgenos por producto, o solo nombre y precio?

R: Únicamente manejamos nombre y precio.

- ¿El menú cambia por temporada o es fijo todo el año?

R: Tenemos un menú fijo.

2.1.3 Flujo de comandas

- ¿Cómo levantan hoy en día una comanda? (papel, apps sueltas, ninguno)

R: Los meseros usan comandas de papel y las entregan con el responsable en caja.

- ¿Quién toma la orden: el mesero en la mesa o se centraliza en un punto?

R: Se centraliza en la barra, donde está el responsable de caja.

- ¿La comanda va directo a cocina o bar, o pasa por un supervisor?

R: Pasa por el responsable en caja.

- ¿Se necesita un estado para la comida? (pendiente, en preparación, lista, entregada, cancelada)

R: Sí.

- ¿Puede una mesa hacer varias rondas de pedidos durante un evento?

R: Sí, un cliente puede pedir varias veces durante un evento.

- ¿Se registran notas especiales por pedido?

R: De ser necesario, sí.

- ¿Quién puede cancelar o modificar una comanda ya enviada a cocina o bar?

R: Encargado de caja.

2.1.4 Cobros y pagos

- ¿El cobro se hace al organizador del evento en una sola cuenta, o se divide por mesa, por invitado o por grupo de invitados?

R: Se puede manejar tanto como un solo cobro al organizador, un cobro por mesa, por invitado o por grupo de invitados. Aunque lo más común es cobrar por grupo de invitados, de los cuales suele haber 2 o 3 por mesa (2 a 3 cuentas por mesa).

- ¿Manejan anticipos o depósitos antes del evento?

R: Sí.

- ¿Qué formas de pago se aceptan y necesitan quedar registradas?

R: Se aceptan pagos en efectivo, transferencia o tarjeta.

- ¿Se requiere factura fiscal (CFDI) o solo un recibo interno?

R: No.

- ¿Hay productos incluidos en el paquete que no se cobren aparte?

R: Únicamente la comida preparada con antelación.

- De ser así, ¿el consumo extra sí se cobra?

R: Sí, el consumo extra a la carta se cobra.

2.1.5 Roles y permisos

- ¿Qué roles existen y qué debe poder hacer cada uno en el sistema? (mesero, cocina, bar, cajero, administrador)

R: Hay mesero, cocina, bar y cajero/administrador.

- ¿Se necesita saber qué mesero levantó cada comanda?

R: Sí, es necesario saber qué mesero hizo qué orden y qué cuentas lleva.

2.1.6 Reportes

- ¿Qué reportes necesita el negocio? (ventas por evento, estadísticas por producto, tiempo promedio de preparación, ingresos por periodo)

R: Serían de utilidad, aunque no necesarios, las ventas por evento y los ingresos por evento.

- ¿Necesita comparar el consumo real contra lo presupuestado en el paquete del evento?

R: Sí.

- ¿Con qué frecuencia se consultan estos reportes? (durante el evento en vivo o solo en el cierre)

R: En su mayoría en el cierre, pero también durante el evento en vivo.

2.1.7 Operación y escalabilidad

- ¿Cuántos eventos atienden en promedio al mes y cuántas comandas por evento?

R: Alrededor de quince eventos mensuales, y en un evento normal suelen ser entre 1 y 3 pedidos por cuenta, pero esto se puede llegar a extender.

- ¿El sistema debe funcionar sin internet dentro del salón, o siempre tienen conexión?

R: Preferentemente que funcionase sin internet.

- ¿Planean usar el sistema en otra sucursal a futuro?

R: No por el momento.

2.2 Requerimientos de la base de datos

2.2.1 Entidades y atributos

- **Evento:** organizador/cliente, fecha, tipo de evento, número de mesas, paquete contratado, estado del evento.
- **Mesa:** número de mesa.
- **Cuenta:** identificador, estado (abierta/cerrada), total acumulado.
- **Mesero / Usuario:** nombre, rol (mesero, cocina, bar, cajero/administrador).
- **Producto:** nombre, categoría (alimentos, bebidas con alcohol, bebidas sin alcohol, dulcería), precio, variantes (tamaño o preparación distinta), ingredientes.
- **Comanda** (pedido): fecha/hora, estado (pendiente, en preparación, lista, entregada, cancelada), detalles.
- **Pago:** monto, forma de pago (efectivo, transferencia, tarjeta), fecha.
- **Anticipo:** monto, fecha.
- **Paquete del evento:** monto presupuestado, productos incluidos.

2.2.2 Relaciones

- **Mesa — Mesero/Usuario:** un mesero atiende una o varias mesas; una mesa puede ser atendida por más de un mesero.
- **Mesa — Cuenta:** una mesa puede tener varias cuentas (grupos de invitados).
- **Cuenta — Comanda:** una cuenta genera varias comandas (rondas) durante el evento.
- **Mesero/Usuario — Comanda:** un mesero levanta varias comandas; cada comanda la levanta un solo mesero, pero más usuarios la atienden.
- **Comanda — Producto:** cada comanda hace referencia a un producto.
- **Cuenta — Pago:** una cuenta puede recibir uno o varios pagos.
- **Evento — Anticipo:** un evento puede tener uno o varios anticipos.
- **Paquete del evento — Evento:** un paquete puede estar en varios eventos y cada evento puede tener uno o varios paquetes.
- **Paquete del evento — Producto:** el paquete incluye uno o varios productos.

2.3 Requerimientos del sistema en general

- Registrar un evento junto con sus mesas y asignar mesero(s) a cada una (un solo mesero si el evento es pequeño, varios repartidos por grupos de mesas si es grande).
- Permitir que una misma mesa tenga varias cuentas independientes (grupos de invitados), cada una con su propio consumo y cobro.
- Centralizar la toma de pedidos en caja: el cajero recibe la comanda (hoy en papel) y la envía a cocina/bar.
- Actualizar el estado de cada comanda (pendiente → en preparación → lista → entregada) y permitir su cancelación o modificación aun después de enviada.
- Permitir múltiples rondas de pedidos por cuenta durante el mismo evento.
- Registrar notas especiales por producto dentro de un pedido cuando sea necesario.
- Registrar qué mesero levantó cada comanda y qué cuentas lleva, para control de responsabilidad y propinas.
- Diferenciar accesos por rol: mesero, cocina, bar y cajero/administrador, cada uno con las funciones que le corresponden.
- Registrar anticipos/depositos del evento antes de la fecha, y aplicarlos al cobro final.
- Registrar pagos por cuenta con su forma de pago (efectivo, transferencia o tarjeta); no se requiere generar factura fiscal (CFDI), solo un recibo interno.
- Distinguir entre consumo incluido en el paquete del evento (comida preparada con antelación, no se cobra aparte) y consumo extra a la carta (sí se cobra).
- Generar reportes de ventas e ingresos por evento.
- Generar un reporte comparativo entre el consumo real y lo presupuestado en el paquete del evento.
- Permitir consultar los reportes tanto en vivo durante el evento como al cierre.
- Operar preferentemente sin conexión a internet dentro del salón.
- Soportar el volumen operativo actual: ~15 eventos al mes, hasta ~20 mesas por evento, 2–3 cuentas por mesa y de 1 a 3 pedidos por cuenta (con posibilidad de más rondas).

2.4 Diagrama Entidad-Relación

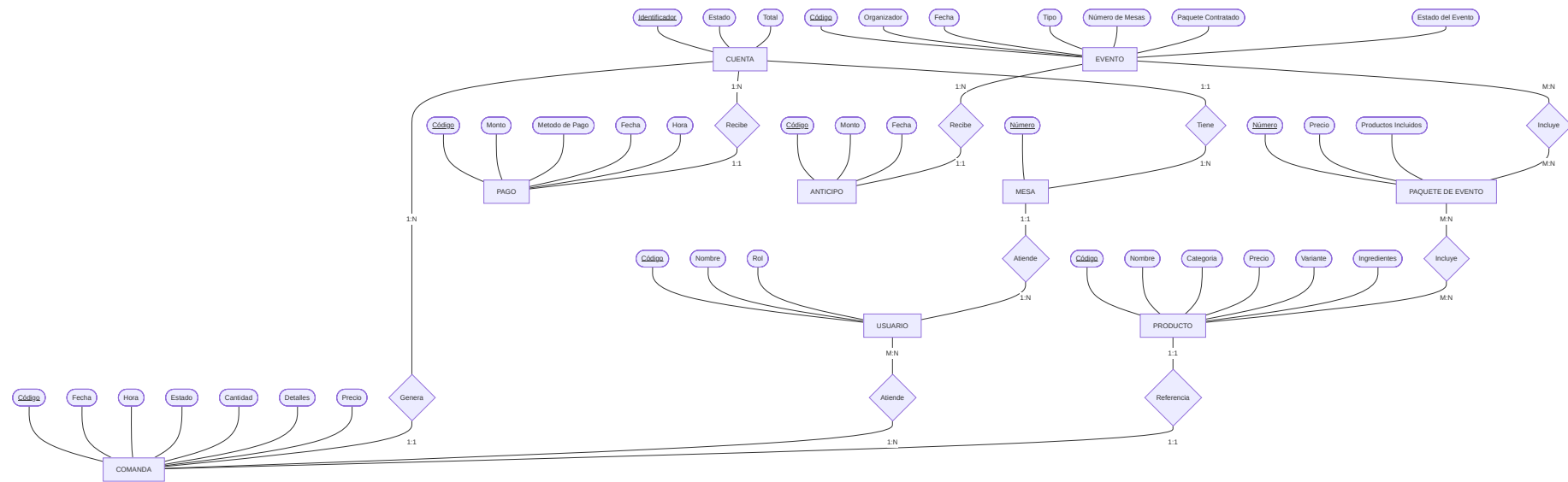


Figura 1: Diagrama entidad-relación

2.5 Justificación del modelo

El modelo entidad-relación de la Figura 1 se construyó a partir de las respuestas obtenidas en la entrevista con el cliente y de los requerimientos que se derivaron de ella. Cada entidad representa un objeto del negocio sobre el cual el salón necesita guardar información de forma permanente, y cada relación representa una regla de operación que el cliente describió. En esta sección se explica por qué se definieron así y de qué manera el diagrama atiende la problemática planteada.

2.5.1 Justificación de las entidades

- **Evento:** es el punto de partida de la operación del salón, ya que el cliente atiende un único evento a la vez y alrededor de quince al mes. Se modela como entidad porque cada evento tiene datos propios (organizador, fecha, tipo, número de mesas y estado) y porque a él se asocian los anticipos y los paquetes contratados. El atributo *estado* permite distinguir un evento programado de uno en curso o ya cerrado.
- **Mesa:** el cliente indicó que las mesas están numeradas y que su cantidad varía entre eventos. Se define como entidad, y no como un simple atributo, porque sobre ella recaen dos reglas del negocio: la asignación de meseros y la división de la cuenta en grupos de invitados.
- **Cuenta:** surge de la respuesta más relevante sobre cobros: lo más común es cobrar por grupo de invitados, con dos o tres cuentas por mesa. Separar la cuenta de la mesa permite que cada grupo tenga su propio consumo, su propio *total* y su propio *estado* (abierta o cerrada), sin mezclarse con el de otros grupos sentados en la misma mesa.
- **Usuario:** agrupa en una sola entidad a todas las personas que operan el sistema. En lugar de crear una entidad distinta para meseros, cocina, bar y caja, se utiliza el atributo *rol*, ya que todos comparten los mismos datos (código y nombre) y solo difieren en las funciones que pueden realizar. Esto simplifica el modelo y facilita agregar roles en el futuro.
- **Producto:** representa los artículos del menú, que el cliente clasificó en alimentos, bebidas con y sin alcohol, y dulcería. Los atributos *categoría* y *variante* responden a que algunos platillos y bebidas se ofrecen en distintos tamaños o preparaciones. Como el menú es fijo durante todo el año, el producto funciona como catálogo estable al que hacen referencia las comandas y los paquetes.
- **Comanda:** es la entidad central del sistema y sustituye a la comanda de papel que hoy se entrega en caja. Registra *fecha*, *hora*, *cantidad*, *detalles* (las notas especiales que el cliente pidió conservar) y *estado*, con el cual se da seguimiento al ciclo pendiente, en preparación, lista, entregada o cancelada. Además guarda el *precio* al momento del pedido: como los precios cambian por temporada, conservarlo en la comanda evita que un cambio posterior en el catálogo altere los totales de eventos ya cobrados, se ah de acla-

rar que el sistema va a manejar cada producto de pedido como una comanda individual para mejorar el flujo de trabajo del sistema.

- **Pago:** el cliente acepta efectivo, transferencia y tarjeta, y necesita que quede registrado. La entidad guarda *monto*, *método de pago*, *fecha* y *hora*, lo que permite saber cómo y cuándo se liquidó cada cuenta. No se incluyen datos fiscales porque el cliente indicó que no requiere factura (CFDI).
- **Anticipo:** se separa del pago porque tiene un origen distinto: se recibe antes del evento y está ligado al evento completo, no a una cuenta de invitados. Mantenerlo aparte evita confundir los depósitos del organizador con los cobros realizados durante el evento.
- **Paquete de evento:** modela la comida preparada con antelación que el organizador contrata y que no se cobra aparte. Registra su *precio* y los *productos incluidos*, lo que establece lo presupuestado para el evento y lo distingue del consumo a la carta.

En todas las entidades se definió un identificador (código, número o identificador, subrayado en el diagrama) como clave primaria, de modo que cada registro pueda distinguirse de manera única aunque otros atributos, como el nombre o la fecha, se repitan.

2.5.2 Justificación de las relaciones

- **Usuario — Atiende — Mesa (1:N):** un mesero puede atender varias mesas. Refleja que, cuando hay varios meseros, a cada uno se le asigna un grupo de mesas, y que en eventos pequeños un solo mesero puede cubrirlas todas.
- **Mesa — Tiene — Cuenta (1:N):** una mesa puede tener varias cuentas, mientras que cada cuenta pertenece a una sola mesa. Es la relación que permite dividir el cobro por grupo de invitados, que es la forma de cobro más común en el salón.
- **Cuenta — Genera — Comanda (1:N):** una cuenta genera varias comandas y cada comanda pertenece a una sola cuenta. Así se representan las distintas rondas de pedidos que un mismo grupo realiza durante el evento, y el total de la cuenta puede obtenerse a partir de sus comandas.
- **Usuario — Atiende — Comanda (M:N):** en una comanda intervienen varios usuarios (el mesero que la levanta, el cajero que la recibe y la cocina o el bar que la preparan), y cada usuario participa en muchas comandas. Esta relación responde a la necesidad expresa del cliente de saber qué mesero hizo cada orden.
- **Comanda — Referencia — Producto (M:1):** cada comanda hace referencia a un producto del catálogo, y la cantidad y el precio pedidos quedan registrados en la propia comanda. Con ello se evita escribir a mano el nombre y el precio del producto en cada pedido.
- **Cuenta — Recibe — Pago (1:N):** una cuenta puede liquidarse con uno o varios pagos,

por ejemplo cuando parte se paga en efectivo y parte con tarjeta; cada pago corresponde a una sola cuenta.

- **Evento — Recibe — Anticipo (1:N):** un evento puede recibir uno o varios anticipos antes de su fecha, y cada anticipo pertenece a un único evento.
- **Evento — Incluye — Paquete de evento (M:N):** un evento puede contratar uno o varios paquetes y un mismo paquete puede ofrecerse en muchos eventos, por lo que los paquetes se definen una sola vez y se reutilizan.
- **Paquete de evento — Incluye — Producto (M:N):** un paquete agrupa varios productos y un producto puede formar parte de varios paquetes. Esta relación deja registrado qué productos están incluidos en cada paquete.

2.5.3 Cómo resuelve el modelo la problemática planteada

La problemática descrita en la introducción se origina en la gestión manual de las comandas y en un sistema anterior basado en únicamente papeles y tablas, sin relaciones bien definidas ni reglas de integridad. El modelo propuesto atiende estos problemas de la siguiente manera:

- **Pérdida y duplicación de pedidos entre meseros, cocina y caja:** la comanda deja de ser una hoja de papel y se convierte en un registro único, con un código propio, que todos los roles consultan. Su atributo *estado* permite a caja, cocina y bar saber en qué punto se encuentra cada pedido sin depender de mensajes verbales.
- **Falta de un registro histórico confiable:** cada comanda queda ligada a la cuenta que la generó, al producto que se pidió y a los usuarios que la atendieron, con fecha, hora y el precio vigente en ese momento. Esto permite reconstruir qué se sirvió, a quién y a qué costo.
- **Cuenta final lenta y propensa a errores:** al separar las cuentas por grupo de invitados y ligar a cada cuenta sus comandas y sus pagos, el total de cada grupo se obtiene directamente de los datos registrados, sin conciliar manualmente varias comandas en papel. Los anticipos, al estar ligados al evento, quedan registrados para descontarse del cobro final.
- **Control de responsabilidad del personal:** la relación entre usuario y comanda deja constancia de qué mesero levantó cada orden y quién más intervino en ella, lo que el cliente señaló como necesario.
- **Diferencia entre lo contratado y lo consumido a la carta:** los paquetes de evento y sus productos incluidos documentan lo que el organizador contrató con antelación, separado de las comandas a la carta que sí se cobran.
- **Deficiencias del sistema anterior:** a diferencia de los archivos ASCII del proyecto previo, cada entidad cuenta con una clave primaria y cada relación tiene una cardinalidad

explícita. Esto permite, en la implementación en PostgreSQL, aplicar atributos fuertes y restricciones de integridad que impidan, por ejemplo, registrar un pago sin cuenta o una comanda que haga referencia a un producto inexistente.

En conjunto, el modelo traslada a una estructura formal las reglas de operación que el cliente describió en la entrevista, y constituye la base sobre la cual se implementará la base de datos del proyecto final.

3 Conclusiones

Referencias

- Aguilar Vera, R., Naal Jácome, A., Díaz Mendoza, J., & Gómez Gómez, O. (2023). NoSQL database modeling and management: A systematic literature review. *Revista Facultad de Ingeniería*, 32(65), Artículo e16519. <https://doi.org/10.19053/01211129.v32.n65.2023.16519>
- Machuca, L., & Braul, R. (2022). Metodologías más usadas en la seguridad de bases de datos: Una revisión de la literatura científica, 2016–2021. *Campus*, 27(33), 121-130. <https://doi.org/10.24265/campus.2022.v27n33.08>
- Taipalus, T. (2024). Database management system performance comparisons: A systematic literature review. *Journal of Systems and Software*, 208, Artículo 111872. <https://doi.org/10.1016/j.jss.2023.111872>

4 Anexos